

08 • Rollouts & scale

Siddharth M. Bhatia · siddharth@sidmb.com

July 1, 2026

Contents

<code>conv.rollout</code> — many runs of one recipe	1
Data-driven rollout — one conversation per dataset row	2
<code>interlens.run</code> — many lineups in one pool	3
<code>analyzer</code> — measure each run while models are resident	4
Multi-GPU: register analyzers by name	5
Matched compute — <code>TokenBudget</code>	5
Large API rollouts (batch mode)	6
Execution modes & reproducibility	7

Run one conversation recipe many times — across many GPUs, checkpointed and resumable — with per-conversation failure isolation (one OOM/bad row can't take down the batch). There is **one object**: a `Conversation` is the recipe, the live dialogue, and the rollout driver. No templates, specs, or configs.

`conv.rollout` — many runs of one recipe

```
from interlens import Conversation, AutoModelParticipant

conv = (Conversation(
    participants=[
        AutoModelParticipant.from_pretrained("Qwen/
        Qwen2.5-3B-Instruct", name="a", temperature=0.9),
        AutoModelParticipant.from_pretrained("Qwen/
        Qwen2.5-3B-Instruct", name="b", temperature=0.9)],
    shared_context="Debate: tabs vs spaces.") # scenario
    framing is a constructor field
    .turns(6)) # rollout/
    data fields use dot-modifier sugar

report = conv.rollout(
    n=64,
    devices=None, # None → all visible GPUs (see
    available_devices()); or e.g. ["cuda:0", "cuda:1"]
```

```

out_dir="runs/tabs_spaces", # checkpoints each conversation →
    resumable
seed=0,                      # per-rollout seeds are seed,
    seed+1, ...
# batched=True is the DEFAULT: local models co-step in one
# batched generate (max throughput). Pass
# batched=False only for the DETERMINISTIC/interp escape hatch
# (see below).
)
for job_id, res in report.results.items():
    if res.error is None:
        print(job_id, "→", res.transcript[-1].content[:80])
    print("failed:", report.failed, "skipped(resumed):", report.skipped)

```

Rollout copies, it does not mutate. rollout makes an independent copy-on-write clone of the conversation per sample/row and runs *those*; the original conv stays the unrun recipe. Finished conversations live on the report — `report.results[job_id].conversation` (with `.transcript`, `.analysis`, `.tokens_generated`). That is where to `sample()` or inspect afterwards, **not** on the original (calling `conv.sample()` on an unrun recipe raises).

Modifiers are copy-on-write too: `conv.turns(6)` returns a *new* conversation, so `conv.turns(12).rollout()` re-runs at a new length without touching `conv`. Use `.set(field=value)` for fields without a dot-modifier.

Data-driven rollout — one conversation per dataset row

A benchmark is *N different scenarios*, not *N copies*. Template a field with `dataset_field` and attach the data — rollout expands to one conversation per row, resolving the field for each:

```

from interlens import Conversation, AutoModelParticipant,
    dataset_field
from datasets import load_dataset

```

```

solver = AutoModelParticipant.from_pretrained("Qwen/Qwen2.5-1.5B-Instruct", name="solver",
                                              system_prompt="Solve
it; end with 'Final answer: <n>'.")
report = (Conversation(participants=[solver],
                      shared_context="Solve this problem.\n\n",
                      dataset_field("question"))) # per-row
            .turns(1)
            .data(load_dataset("openai/gsm8k", "main",
split="test").select(range(200)))
            .analyzer(lambda conv: {"answer":
conv.transcript[-1].content})
            .rollout(out_dir="runs/gsm8k")) # n defaults to
len(data); job ids row_00000...

```

shared_context, shared_system_prompt, and each participant's system_prompt may be templated. A template value is a str (as-is), dataset_field("col"), a callable(row) → str, a tuple of those joined (("Q: ", dataset_field("q"))), or a PEP 750 t-string on Python 3.14+. Resolution happens once at expansion (row $i \longleftrightarrow$ job i), so resume stays correct.

The data is streamed, never materialized — rows are pulled one at a time, so a map-style Dataset stays Arrow-backed on disk and a load_dataset(..., streaming=True) IterableDataset (a corpus too big for memory) works unchanged; n defaults to len(data) when known, else all rows.

Per-row side data reaches the analyzer via conv.row — the dataset row that produced a conversation is available (as a dict) on conv.row, so an analyzer can grade against a label/gold WITHOUT templating it into the model's view (where it would leak). The runnable [examples/gsm8k_benchmark_rollout.py](#) is the full end-to-end pattern — it adds a gold column with a lazy .map and grades conv.row["gold"] in the analyzer.

interlens.run — many lineups in one pool

When you have *different* lineups (a ladder of model pairs × conditions), hand them all to run so GPUs stay packed across lineups (no idle tail between sequential rollouts), under one out_dir/resume namespace:

```

from interlens import run
report = run(
    [conv_a.name("pair_7b"), conv_b.name("solo_7b"),
     conv_c.data(gpqa).name("gpqa")],
    devices=None, out_dir="runs/ladder", resume=True,
) # job ids are namespaced by each conversation's name(): pair_7b,
   solo_7b, gpqa/row_00000, ...

```

Each conversation expands to its own jobs (one per `data()` row, else a single job); all jobs share one pool. Mixing lineups is safe: batched co-stepping groups by **schedule signature** (turns + per-position model identity), so each distinct lineup forms its own batch group and a one-off lineup runs as a singleton. `conv.rollout(...)` is the single-lineup sugar for this.

- **Parallel by default on two axes:** one worker *process per device*, AND *batched co-stepping* within each device. No flag needed.
- **Single device** → runs in this process (the path where an analyzer may be a plain callable/closure).
- **Multiple devices** → one process per GPU (`torch.multiprocessing`, since `fork+CUDA` is broken). Conversations pickle cheaply — lazy participants ship no weights, each worker loads on its own GPU. A closure analyzer won't survive pickling, so **register analyzers by name** for the multi-GPU path (below).

```

RunReport: .results({job_id: RunResult}), .failed, .skipped,
plus .transcripts()/ .analyses()/ .conversations(). RunResult: .conversation, .transcript, .analysis, .

```

analyzer — measure each run while models are resident

An analyzer runs **inside the worker** right after the conversation finishes, while the models are still on-GPU — so it can `sample`, `branch`, or read activations on the live conversation. Only its **serializable return value** is kept (in `RunResult.analysis`). Attach it with `.analyzer(...)`:

```
def summarize(conv):
    return {"n_turns": len(conv.transcript),
            "a_stance": conv.sample("a", "One word: your final
            stance?").content.strip()}

report = conv.analyzer(summarize).rollout(n=32, out_dir="runs/x")
# in-process: a callable is fine
```

Multi-GPU: register analyzers by name

Spawned workers inherit no parent state, so an analyzer that must run in the pool has to be **registered by name at import time** (a closure over parent locals can't cross the process boundary):

```
from interlens import register_analyzer

def stance_probe(conv):
    return {"a": conv.sample("a", "One word
    stance?").content.strip()}

register_analyzer("stance_probe", stance_probe)      # at module
import
report = conv.analyzer("stance_probe").rollout(n=64, devices=None,
out_dir="runs/x")
```

For heavy per-worker setup (e.g. loading a probe/classifier once per GPU), use `register_worker_init`.

Matched compute — TokenBudget

To compare a solo model against a pair *fairly*, give the solo run the same token budget as the pair. `TokenBudget` is a `StopCondition` that stops each conversation once ITS OWN generated tokens hit `per_conversation`, and can spread the budget across turns with `per_turn`:

```
from interlens import TokenBudget

pair = conv
solo = conv.set(participants=[solver]).turns(None).run_until(TokenBudget(per_conversation=pair
per_turn=512))
solo_report = solo.rollout(n=64)
```

```
# or ambiently over any run/rollout in the block – the budget is PER
# conversation, not a shared pool:
with TokenBudget(per_conversation=2048):
    conv.rollout(n=200) # every one of the 200
                        # conversations gets its own 2048-token budget
```

Counting is free (summed from each turn's recorded `metadata['n_tokens']`, never re-tokenized) and per-conversation (read from the conversation's own transcript). Verify realized spend off the report: `report.results[jid].tokens_generated`.

Large API rollouts (batch mode)

For hosted models (`APIParticipant`), set `batch=True` to route each round's turns through the provider's **asynchronous batch API** — ~50% cost and much higher throughput, at the price of batch-window latency:

```
from interlens import Conversation, APIParticipant
report = (Conversation(participants=[
    APIParticipant(name="pro", provider="openai",
        model_id="gpt-5", batch=True, system_prompt="Argue
        YES."),
    APIParticipant(name="con", provider="anthropic",
        model_id="claude-sonnet-5", batch=True, system_prompt="Argue
        NO.")]
    , shared_context="Is a hot dog a sandwich?")
    .turns(4)
    .rollout(n=200)) # each round → one Anthropic/OpenAI
                    batch per speaker
```

- **Batch APIs exist only for `provider="anthropic"` and `provider="openai"`.** Requesting `batch=True` on `provider="openrouter"` **raises** (OpenRouter has no batch endpoint) — it never silently degrades to serial calls.
- With `batch=False` (default) API turns run as ordinary concurrent single requests, bounded by the shared client's max-in-flight semaphore.

See [participants & models](#) for provider setup + API keys.

Execution modes & reproducibility

- **Batched co-stepping is the default** (`batched=True`): local `ModelParticipants` (every family) co-step, trading exact per-row reproducibility for throughput — batch composition + a single global RNG perturb rows, so only *distributional* reproducibility holds. Batched turns are marked in `metadata["batched"]`; the shared-prefill fast path (turn 1 of an n-sample rollout off one scenario) in `metadata["shared_prefill"]`.
- `batched=False` is the `ExecutionMode.DETERMINISTIC` escape hatch: each rollout runs independently and is **token-identical** on the same hardware (pin per-participant `seed=` and `kv_reuse=False`). It is the only path that supports per-turn interp (capture/steering/probes) — batched generation cannot.

Next: [advanced interp pipelines](#).